ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/374147236

Detecting face masks through embedded machine learning algorithms: A transfer learning approach for affordable microcontrollers

Article *in* Machine Learning with Applications · September 2023

DOI: 10.1016/j.mlwa.2023.100498

CITATIONS
3

READS
143

5 authors, including:



Thaís de Araújo de Medeiros
Federal University of Rio Grande do Norte
2 PUBLICATIONS **3** CITATIONS
SEE PROFILE



Morsinaldo Medeiros
Federal University of Rio Grande do Norte
13 PUBLICATIONS **21** CITATIONS
SEE PROFILE



Ivanovitch Silva
Federal University of Rio Grande do Norte
145 PUBLICATIONS **1,959** CITATIONS
SEE PROFILE



Daniel G. Costa
University of Porto
189 PUBLICATIONS **1,952** CITATIONS
SEE PROFILE

All content following this page was uploaded by Daniel G. Costa on 02 October 2023.

The user has requested enhancement of the downloaded file.

Machine Learning with Applications 14 (2023) 100498

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

Detecting face masks through embedded machine learning algorithms: A transfer learning approach for affordable microcontrollers

Mariana B. Azevedo [a,b], Thaís de A. de Medeiros [a,b], Morsinaldo de A. Medeiros [a,b], Ivanovitch Silva [a,b], Daniel G. Costa [c,*]

[a] Graduate Program in Electrical and Computer Engineering (PPgEEC), Federal University of Rio Grande do Norte, Natal/RN, 59078-970, Brazil
[b] Department of Computing Engineering and Automation (DCA), Federal University of Rio Grande do Norte, Natal/RN, 59078-970, Brazil
[c] INEGI, Faculty of Engineering, University of Porto, Porto, 4200-465, Portugal

ARTICLE INFO

Keywords:
Embedded Machine Learning
TinyML
Mask detection
Transfer Learning
Edge computing

ABSTRACT

The adoption of face masks to halt the spread of airborne diseases has been recognized as an effective measure, being a significant resource to reduce new infections of different types of virus that affect the respiratory system. In this sense, the enforcement of the use of masks in crowded areas has been a recurrent concern of governments, which has fostered the development of camera-based approaches to automatically detect people without face masks. In order to tackle this problem, this article proposes a low-cost approach based on the use of affordable and energy-efficient microcontrollers to quickly process images and detect when people are not wearing face masks, performing local decisions. For this on-the-edge processing, reduced Machine Learning models were created and evaluated through different techniques. As a result, a complete processing pipeline was designed to perform a transfer learning process taking as reference the MobileNet model supported by the Edge Impulse platform, achieving an innovative TinyML solution for quick decisions on resource-constrained Internet of Things units. Additionally, we performed a series of experiments on the Arduino Nano 33 board attached to an OV7675 camera module, evaluating the practical application of the proposed solution after proper training using public image datasets. Working as a soft-sensor unit that is reproducible and ready to be used, the defined trainable model and the performed evaluation results represent a significant contribution towards real-world deployment of face masks detection systems.

1. Introduction

The development of new technologies for performing efficient and configurable image processing has favored a broad range of applications, especially when pursuing the detection of patterns to support decision-making procedures (Ker, Wang, Rao, & Lim, 2017; Li, Zhang, Xue, Jiang, & Shen, 2018). In this context, algorithms for extracting relevant information from images have evolved significantly in recent years. However, issues such as required processing resources and maximum operation time have been crucial for certain applications (Revina & Emmanuel, 2021; Zafeiriou, Zhang, & Zhang, 2015). Consequently, new alternatives have emerged to achieve efficient image processing, with Machine Learning (ML) algorithms being considered the most suitable candidates for processing images in numerous scenarios (Dargan, Kumar, Ayyagari, & Kumar, 2020; Minaee et al., 2021; Wang, Chen, & Hoi, 2020).

With the dramatic events resulted from the COVID-19 pandemic and the urgent necessity for mechanisms to curb virus spread, distributed efficient image processing has gained momentum (Budd et al., 2020; Costa & Peixoto, 2020; Mbunge, Akinnuwesi, Fashoto, Metfula, & Mashwama, 2021). Suddenly, various promising approaches have emerged to ensure social distancing, track infected individuals, and enforce the proper use of face masks, leveraging different technologies (Seker, Männistö, Iosifidis, & Raitoharju, 2022). However, concerning the specific case of face mask detection based on image processing, conventional approaches have utilized algorithms that require substantial processing and memory resources (Kumar, Kaur, & Kumar, 2019), potentially limiting their scalability. Consequently, ensuring efficient levels of face mask detection while necessitating reduced hardware resources presents a significant research gap to overcome. Additionally, although the COVID-19 pandemic has largely receded from global headlines and is no longer a foremost concern for worldwide health systems, the emergence of new airborne viruses and mutations remains inevitable, consistently demanding novel, efficient, and cost-effective measures to impede their rapid transmission.

* Corresponding author.
E-mail addresses: mariana.brito.110@ufrn.edu.br (M.B. Azevedo), thais.araujo.707@ufrn.edu.br (T. de A. de Medeiros), morsinaldo.medeiros.075@ufrn.edu.br (M. de A. Medeiros), ivanovitch.silva@ufrn.br (I. Silva), danielgcosta@fe.up.pt (D.G. Costa).

https://doi.org/10.1016/j.mlwa.2023.100498
Received 27 June 2023; Received in revised form 2 September 2023; Accepted 16 September 2023
Available online 23 September 2023
2666-8270/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

In recent years, a new generation of small multi-purpose microcontrollers with reasonable processing power and memory capabilities has become available at affordable prices (Ojo, Giordano, Procissi, & Seitanidis, 2018). When coupled with cost-effective cameras for real-time image retrieval (Costa, 2020), these microcontrollers can be harnessed to identify various patterns, such as proper face mask usage, at a minimal expense. The challenge has been to incorporate machine learning models into such hardware units, as the available processing power is significantly lower than what conventional computers can provide (Pham, Nguyen, & Sun, 2019; Sharifi, Zibaei, & Rezaei, 2021). These resulting compact machine learning models are referred to as TinyML (Andrade et al., 2022; Warden & Situnayake, 2020).

This article presents a soft-sensor TinyML-based approach to detect face masks through the processing of still images (photographs) of individuals passing through a designated area. The approach is implemented on a microcontroller connected to a low-resolution camera and a motion sensor. The intention is to make decisions at the edge, potentially improving the overall performance of face mask detection systems through localized decisions. To attain this objective, we employed the pre-trained MobileNet ML architecture, alongside feature extraction and fine-tuning techniques (Howard et al., 2017; Kaya & Gürsoy, 2023). A transfer learning process utilizing the MobileNet architecture was executed on the Edge Impulse platform (Hymel et al., 2022) to achieve a deployable TinyML model. This model enables the implementation of a real-world solution on the Arduino Nano 33 BLE Sense board, coupled with an OV7675 camera module.

To ensure comprehensive coverage of the entire development process leading to the final solution, this article examines and compares the performance of various potential TinyML architectures. Specifically, we focus on VGG16, DenseNet, and MobileNet, evaluating their suitability for the defined scope of the problem. This analysis provides valuable insights and contributes to the research addressing this specific problem. Additionally, we assess the outcomes of incorporating feature extraction and fine-tuning techniques. This evaluation offers valuable insights into the anticipated performance across different configurations.

Finally, a comprehensive performance evaluation will be conducted to demonstrate the effectiveness of our approach when detecting face mask, considering both data analysis and a real deployment scenario. Since the proposed approach is intended to work as a soft sensor that is ready to be deployed in areas with a flow of people (usually going in the same direction, such as in airports and metro stations, for example), this evaluation process wants to assess the effectiveness when properly detecting face masks, as well as its hardware demands and expected energy consumption. The complete developed solution, along with the code for the implemented evaluation prototype, is available for open access at Azevedo, Medeiros, Medeiros, Silva, and Costa (2023).

The remainder of this article is organized as follows. Section 2 brings recent works that influenced our research, highlighting their main differences. The development of the proposed approach is described in Section 3. Section 4 presents the achieved performance results and evaluation analysis, followed by discussions of potential applications for the proposed methodology in Section 5. Finally, conclusions and references are presented.

2. Related works

TinyML refers to the implementation of machine learning algorithms on small, low-power devices such as microcontrollers and sensor-equipped nodes. These devices are commonly utilized in Internet of Things (IoT) applications, and integrating machine learning algorithms into them brings forth an array of capabilities, including enhanced sensor data analysis, on-device decision-making, and improved power efficiency. With the COVID-19 pandemic, several works have been conducted to exploit embedded machine learning techniques to support in different actions to both halt the spread of the virus and to relieve its negative impacts. In fact, the achieved contributions opened an important development trend that will somehow impact how we face new respiratory diseases outbreaks and eventual pandemics. In this context, a subset of practical solutions have emerged to detect whether or not an individual is wearing a face mask to reduce the spread of a virus, with some of them being discussed in this section.

The work in Said (2020) developed an optimized approach based on a Convolutional Neural Network (CNN) for use in low-power devices, with a focus on high performance. To achieve this, that work employed the YOLO framework and combined multiple datasets for training, resulting in a dataset of over 500,000 images that included people both wearing and not wearing face masks. Through this, a precision of 94.6% was achieved. The model is then further optimized, retrained, and embedded on the PYNQ-Z1 board, resulting in a precision of 90.7%.

Authors in Yasin Kaya (2023) developed a CNN model based on the MobileNetV2 architecture that aims to classify whether a lung is healthy, infected with pneumonia, or with COVID-19, using chest X-ray images as input. For this purpose, a database containing 3616 COVID-19-related images, 1576 images of healthy lungs, and 4265 images with pneumonia was utilized. This totaled 9457 images, which were pre-processed and resized to 224 × 224 dimensions. Furthermore, data augmentation techniques such as image rotation, random horizontal flipping, and zooming were employed to fine-tune six pre-trained models: VGG16, InceptionV3, MobileNetV2, ResNet50, DenseNet, and EfficientNet. The validation accuracy was 0.6250, 0.9375, 0.9549, 0.9488, 0.9375, and 0.8346, respectively, demonstrating that the MobileNetV2 architecture achieved the highest accuracy in comparison to the other models. Finally, it is essential to note that the authors' solution targeted more potent hardware. To implement it within the TinyML context, techniques such as quantization and pruning would be necessary to reduce the model's size.

In Raza, Osman, Ferrini, and Natale (2021), authors proposed utilizing edge deep learning to address the issue of flight time limitations in Unmanned Aerial Vehicles (UAVs). Their solution involves integrating an OpenMV microcontroller into a DJI Tello Micro Aerial Vehicle to control the drone's navigation and accomplish tasks. To validate the approach, the proposed practical application involves detecting individuals wearing protective masks in a crowded environment. For this purpose, the Face Mask Lite open-source dataset was adopted for training and validation, with 100,000 images randomly selected, encompassing both face mask wearers and non-wearers. Each image was resized to 96 × 96 pixels. Furthermore, the model utilized was based on the MobileNetV2 architecture. Implementations of the trained model were compared between the Arduino Nano 33 and OpenMV platforms. The results demonstrated that live classification on the Arduino Nano 33 encountered challenges due to RAM limitations, necessitating 440.4 KB and 7 s for inference, with an estimated accuracy of 97%. The OpenMV live classification took 859 ms, and RAM usage did not reach maximum capacity, yielding an accuracy of 98.5%.

In Piatkowski and Walkowiak (2022), authors employed the sliding window technique in conjunction with the k-means classification algorithm, integrated into a BlackPill board featuring the STM32F411 microcontroller. This approach aims to reduce the computational cost from a nested loop to that of a single loop, demonstrating notable efficiency in computer vision tasks. Additionally, the work in Piatkowski and Walkowiak (2022) highlights that the system was designed to function with a camera capturing 640 × 480 images. These images are subsequently downscaled to 32 × 32 resolution and converted to grayscale. Following comparisons with Support Vector Machines and Neural Networks, the authors opted for the K-means algorithm due to its lower computational complexity among the three options. As a result, the K-means algorithm achieved approximately 79.51% accuracy in detecting masked faces.

In Mohan, Paul, and Chirani (2021), the authors proposed an architecture based on CNN and trained it with TensorFlow Lite, aiming to detect medical facial masks for deployment in endpoints with limited

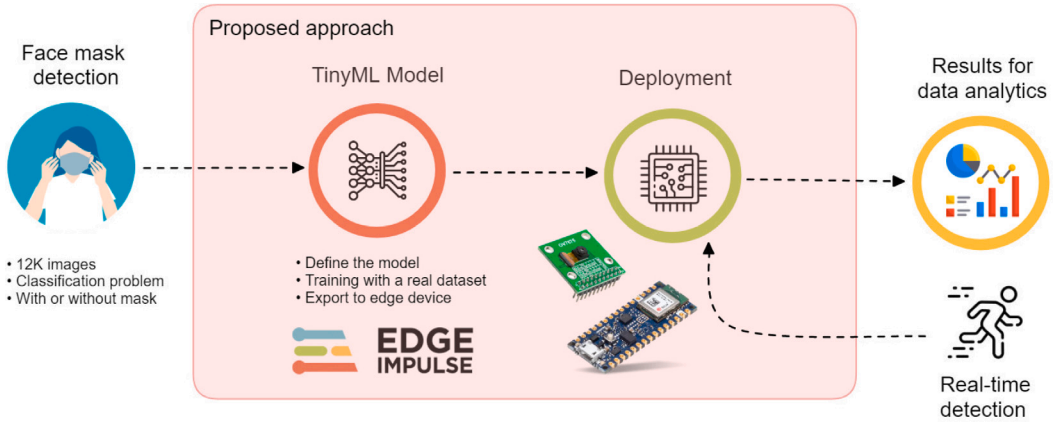


Fig. 1. The conceptual elements of the proposed approach.

Table 1
Previous works exploiting TinyML to process visual data for classification.

Work	Year	AI models	Hardware
Said	2020	Pynq-YOLO-Net	Pynq Z1
Raza et al.	2021	MobileNetV2	Arduino Nano 33, OpenMV H7
Mohan et al.	2021	SqueezeNet, SqueezeNet Modified, CNN-based	OpenMV H7
Piatkowski and Walkowiak	2022	K-means	BlackPill
Yasin Kaya	2023	MobileNetV2, VGG16, InceptionV3, ResNet50, DenseNet, EfficientNet	Not speficied

resources. To achieve this, they employed various datasets that, following data augmentation, aggregated to 131,055 images. These images were resized to 32 × 32 pixels and normalized for model training. The training was conducted using TensorFlow Lite, targeting an OpenMV Cam H7 microcontroller, which boasts more computational power compared to the Arduino Nano 33 board. For inference, the authors utilized the OV7725 camera alongside a script that captures 200 × 200 pixel images, downscales them to 32 × 32 pixels, normalizes them, and performs inference. Moreover, model quantization from Float −32 to Int-8 was applied, reducing the model size from 1.52 MB to 138 KB and elevating the test accuracy from 99.81% to 99.83%. Finally, the authors noted that their hardware achieves an inference rate of 30 Frames per Second (FPS), translating to approximately 33 ms per inference.

Table 1 summarizes the reviewed works, pointing out their main characteristics.

This article presents a solution using CNN and transfer learning based on the MobileNetV1 architecture and embedded on an Arduino Nano 33 BLE Sense board aiming to classify if, given an image, a person is wearing a mask or not. To achieve this, the optimal Machine Learning Operations (MLOps) practices for embedded devices were employed through the Edge Impulse platform, which allows an AutoML to be performed using the EON Tuner tool, and thus choosing the best model to be embedded in the target device. Furthermore, the established development pipeline ensures the reproducibility of all executed steps. By combining these elements, the proposed approach potentially offers greater efficiency, affordability, and ease of implementation compared to prior works, thereby contributing significantly to this field.

As an additional comment, while we agree that enhancing the clarity of our work's contributions through a comparison with existing methods might be relevant, we find that this comparison is constrained by significant factors. Notably, our proposed TinyML pipeline constitutes a comprehensive solution development process, encompassing essential hyperparameters often underrepresented in prior literature. Unfortunately, the absence of detailed equivalent information in previous works impair a meaningful comparison. Moreover, our emphasis on affordability, achieved by employing economical microcontrollers and sensors, distinguishes our approach from the majority of preceding methods, especially in the domain of face mask detection. This critical dimension has been notably overlooked in earlier methodologies addressing TinyML-based face mask detection. Furthermore, our commitment to reproducibility is demonstrated by the provision of complete solution code, which can be openly and freely accessed at Azevedo et al. (2023). This practice, however, remains uncommon in prior studies within this field, hampering equitable comparisons. Therefore, given these constraints, a direct comparison with existing approaches would lack both contextual parity and substantive insights. Instead, we advocate for highlighting these distinctive elements individually, thus emphasizing their substantial contributions to the manuscript, as performed in this section.

3. Proposed approach

This section presents the employed methods when designing and implementing the proposed approach, describing the details about the adopted images dataset, the chosen ML architecture, the training process, and the performed validations. Fig. 2 extends the conceptual model previously described in Fig. 1, highlighting the general pipeline when creating and validating the final proposed TinyML model.

It is worth mentioning that the Edge Impulse platform was used as a core element of the adopted methodology, but it has to be considered as just a tool to accomplish the expected goals. Actually, the defined development pipeline is a consistent general approach to develop robust TinyML-based solutions, and it is reasonable to expect that any supporting tool could be employed in this process. Nevertheless, since the Edge Impulse is a MLOps platform that aims to allow the efficient development of TinyML-based systems for free, supporting the embedding of AI-based solutions into several types of hardware (Hymel et al., 2022), we believe that better results could be achieved while reducing the overall development time and related costs.

Overall, the specification and practical exploitation of our proposed TinyML pipeline may be somehow complex, encompassing a great number of hyperparameters. Although next subsections will focus on the major architectural elements through the defined processing steps, all data and metadata associated to our proposed approach, comprising both processing methodology and training/validation dataset, can be openly and freely accessed in Azevedo et al. (2023). Moreover, additional diagrams and conceptual figures concerning our interaction with the Edge Impulse platform are also included there. Readers are encouraged to consult that code for additional details.

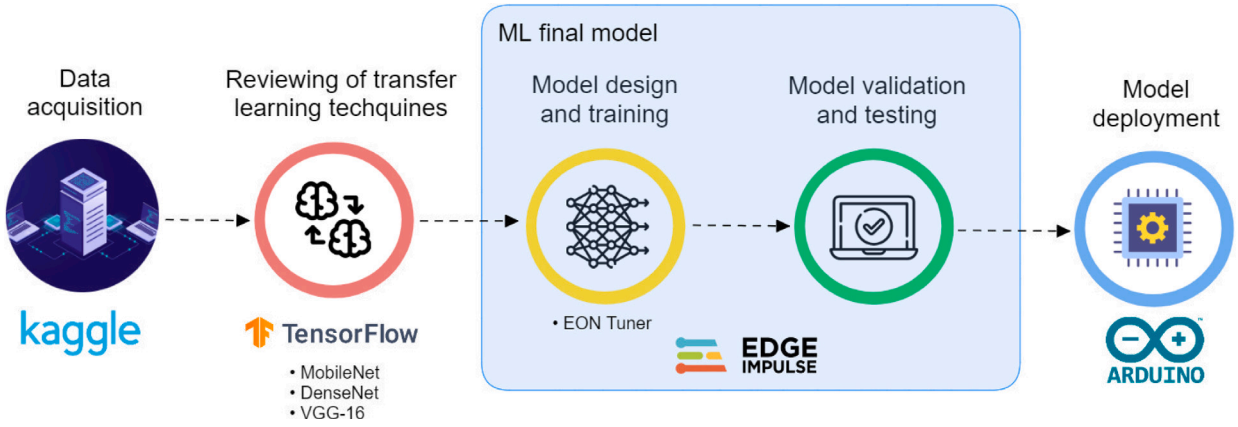


Fig. 2. The methodological pipeline when creating the TinyML-based soft-sensor approach. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.1. Data acquisition

For machine learning approaches that are based on a training process, the acquisition and processing of data is a key element of the developed solution. In this article, the input data for training was sourced from an online dataset available on Kaggle (Bojer & Meldgaard, 2021). This database contains approximately 12 thousand images in PNG format (with different resolutions) of people using or not face masks. Among the images present on the dataset, there are individuals of the most diverse shapes, varying in age, gender and race, as well as head position, mask position, and even mask color. Such images were downloaded with their respective labels (*with_mask* and *without_mask*), and then, they were organized and inserted into the Edge Impulse platform.

To upload the images onto the platform, they were categorized into two distinct folders, each aligning with its respective labels. This categorization was implemented due to the platform's inherent capacity to automatically allocate data into training, validation, and test subsets. Consequently, within the data acquisition section of Edge Impulse, the images were uploaded via the "upload data" option, with corresponding labels assigned to both classes of the classification problem.

Then, still within this step related to data acquisition, a data segregation process was performed. Automatically, the Edge Impulse platform already separates the data in a way that 80% are for training and 20% are for testing and validation, which is a very common data-splitting strategy (Joseph, 2022). However, sometimes this separation may be not well balanced, so a new data segregation was performed selecting the option to perform a new train/test split (potentially reducing bias).

At the end of this process, the total 7298 images that were uploaded to the platform were divided so that 5837 were intended for training and 1461 were intended for testing, with the *with_mask* and *without_mask* classes distributed in a balanced way.

Finally, a brief data analysis was performed using the platform's data explorer functionality. For this, a pre-trained model was selected with the dimensionality reduction technique called t-SNE, a variant of the Stochastic Neighbor Embedding technique that reduces the amount of points in the center of a graph, with potential great results (van der Maaten & Hinton, 2008).

As a final comment, the quality of the performed inferences will depend on the overall quality (diversity, reduced bias, accuracy, data representativeness, among others) of the employed training dataset. Even when not considering the negative effects of data drifting (Suárez-Cetrulo, Quintana, & Cervantes, 2023), the created model may perform poorly depending on the data used for training. As an example in our context, a reduced number of people with glasses of thick beards within the dataset can compromise the accuracy of the model when people with these characteristics are considered. This way, we will consider other datasets in future developments, enhancing the quality of our model. Moreover, reinforcing learning will be considered as an important development step, allowing the use of images taken after physical deployment for new training phases. Although such data-centered improvements are not reflected in this work, they will be updated in the public repository in Azevedo et al. (2023).

3.2. Transfer learning

An important step of the defined methodology is the selection of a proper AI architecture that will be used as a model basis for the intended classifier, as indicated in the pipeline in Fig. 2. At this point, there are several architectures that could be adopted as reference, potentially leading to different results in terms of accuracy and processing performance. Concerning the current state of the art in this area, specially for quick image classification, three transfer learning architectures were chosen to be evaluated concerning their suitability for the mask classification problem: VGG-16, DenseNet, and MobileNet. Since transfer learning is related to the ability to reuse already existing models as a starting point, facilitating the creation of new models at a very low cost, this paradigm was adopted in this work as a development reference.

3.2.1. VGG-16 architecture

The VGG-16 architecture, also known as ConvNet, was proposed by Karen Simonyan and Andrew Zisserman in 2014 (Simonyan & Zisserman, 2014), being the winner of the Imagenet competition (Denton, Hanna, Amironesei, Smart, & Nicole, 2021) of the same year, strongly indicating its high robustness to perform image classification.

Structured in a convolutional design, its pivotal characteristic lies in the consecutive utilization of multiple 3X3 kernel-sized filters. This sequential deployment fosters enhanced neural network depth, facilitating the acquisition of more intricate features while incurring a modest computational expense (Sarker, Islam, Hannan, & Ahmed, 2020). Consequently, the architecture comprises 16 layers and encompasses a total of 134 million parameters (Simonyan & Zisserman, 2014), rendering its implementation on resource-constrained devices more challenging.

3.2.2. DenseNet architecture

The DenseNet is a convolutional architecture that proposes to connect all network layers directly to each other, with different corresponding feature-maps (Huang, Liu, Van Der Maaten, & Weinberger, 2017), thus characterizing a dense connection. This characteristic of the DenseNet architecture makes its models more compact, and consequently, less prone to overfitting problems (Zhu & Newsam, 2017).

In general, this architecture trains approximately 7.8 million parameters, less than the VCG-16 architecture.

Table 2
Results from tests made in the Google Colaboratory platform. Number of samples used for testing: 471.

Architecture	Accuracy	F1-Score	Precision	Recall
VGG-16 [feature extractor]	0.9955	0.9955	0.9956	0.9955
VGG-16 [fine-tuning]	0.9978	0.9978	0.9978	0.9978
DenseNet [feature extractor]	1.0000	1.0000	1.0000	1.0000
DenseNet [fine-tuning]	0.9866	0.9866	0.9870	0.9866
MobileNet [feature extractor]	0.9933	0.9933	0.9933	0.9933
MobileNet [fine-tuning]	0.9978	0.9978	0.9978	0.9978

3.2.3. MobileNet architecture

The MobileNet architecture, created by Google, is a smaller convolutional neural network architecture with fewer parameters, ideal for mobile and embedded computer vision applications, since it makes the most of computational resources without decreasing the model accuracy (Nan, Ju, Hua, Zhang, & Wang, 2022).

Essentially, this architecture is based on the use of convolution layers separated in depth in almost all of its layers (Howard et al., 2017). Actually, this feature is responsible for decreasing the size and complexity of the model. This architecture trains approximately 4.2 million parameters, even less than the DenseNet architecture, being an indication that it is the best transfer learning architecture to be used in the embedded mask detection problem.

3.2.4. Selecting the transfer learning architecture

After highlighting the main characteristics of the candidate architectures, some tests were made on the Google Colaboratory platform using the libraries Tensorflow and Keras in order to validate which of them would be the best fit for building the final TinyML model for our target problem. For this process, a total of six different tests were performed, in which each architecture was trained using two different transfer learning techniques: feature extractor (Abdullah & Hasan, 2017) and fine-tuning (Cetinic, Lipic, & Grgic, 2018). Doing so, the final model would be the one that performed better for the available training dataset.

Table 2 shows the results obtained in those tests. Based on the achieved results, the MobileNet architecture was chosen for three main reasons. In first place, it is the pre-trained model with the least number of parameters, being ideal for adoption by an embedded application. Secondly, it proved to have classification results as good as the other architectures, which are indeed much more processing-demanding considering the training costs. And finally, it was also the architecture suggested by the Edge Impulse EON tuner tool, further validating the choice of this architecture as being the best option for the defined classification problem.

As an additional comment, the choice of the parameters for the evaluation and comparison procedures concerning the pretrained CNN models was based on existing literature recommendations. We chose to adopt values that are widely accepted and suggested in the literature, as they have consistently proven effective across various scenarios and datasets. Such parameters can be also consulted in the Github repository associated with our work, specifically within the Jupyter Notebook files in Azevedo et al. (2023).

3.3. Tinyml final model

In order to design the TinyML model responsible for classifying people wearing or not face masks, it was used the EON Tuner tool from the Edge Impulse platform, taking as reference the previously selected MobileNet architecture. This tool aims to promote automated machine learning (AutoML) techniques to create models as well as the best choices of hyperparameters for these models (hyperparameter tuning), taking into account the specifications of the hardware on which the solution will be embedded (Hymel et al., 2022).

Initially, it is necessary to indicate which will be the target device, as well as the latency time. For our approach, as mentioned earlier, it was used the Arduino Nano 33, and the latency time was initially set at 1000 ms (which is a "desired" latency time for inferences in average). Moreover, among the main parameters used to perform the hyperparameter tuning, there are other options: the image dimension, the image color (RGB or grayscale), the learning rate, the number of training cycles, and the neural network architecture. At this point, we checked the available architectures again, trying to confirm whether the MobileNet architecture was the most adequate for the learning transfer process.

After initiating the configuration procedure, the platform suggested 10 different models to build the neural network architecture of the classifier. Among them, the model with the highest accuracy was the MobileNet (95%), even when considering the RAM and ROM (Flash) usage, as well as latency time. Thus, after this second round of evaluation, the MobileNet architecture was confirmed as the chosen reference model for the proposed approach.

Moreover, in addition to the selection of the MobileNet architecture, the complete design of the model comprises a dense layer with 64 neurons, a dropout layer with a threshold value of 0.1, and finally an output layer that will perform the final classification. Therefore, the processing of the input data passes through all this elements, sequentially, performing the expected computations.

Regarding the chosen hyperparameters, the model input consists of images with dimensions of 96×96 pixels, employing the RGB color pattern. In addition, 20 training cycles were performed and a learning rate of 0.0005 was established.

Finally, about the employed hardware, it is important to point out that the Arduino Nano 33 BLE Sense board uses a 64 MHz nRF52840 processor with 256 kB RAM and 1024 kB Flash memory (Arduino docs, 2023). In the chosen model, such hardware constraints had to be properly considered, as discussed in next section. Regarding latency, a maximum value of 1191 ms was obtained, slightly higher than the desired value of 1000 ms, but it is still a very fast time to perform the intended inference in most application scenarios.

4. Results and discussions

After achieving a TinyML model to tackle the problem of efficient detection of face masks, which was suitable to be embedded into a cost-effective solution, a series of tests could be performed as a final validation stage. Actually, according to Fig. 2, the achieved model was validated through test procedures, evaluating its effectiveness for the set of test images. After that, the deployment of the model could be performed, allowing the implementation of a prototype unit to finally validate the designed approach. All these steps are discussed in the following subsections.

4.1. Validation and testing

The considered datasets for training and testing were employed in different stages of the proposed approach. As mentioned earlier, 80% of the available data was employed during the training phase, with the remaining 20% being used for testing. Considering these datasets, we initially evaluated the overall distributions with a feature explorer procedure, showing the entire group of input data as a 2D graph. Such two-dimensional graph is presented in Fig. 3, depicting that very few images are grouped in the incorrect class using this visualization technique.

In general, images classified as $without_mask$ that are in the $with_mask$ class cluster are images of people with some object near their nose and mouth, or men with a large beard. The opposite situation occurs with images of people with the mask covering only the mouth, or in situations where the color of the mask is very similar to the individual's skin color.

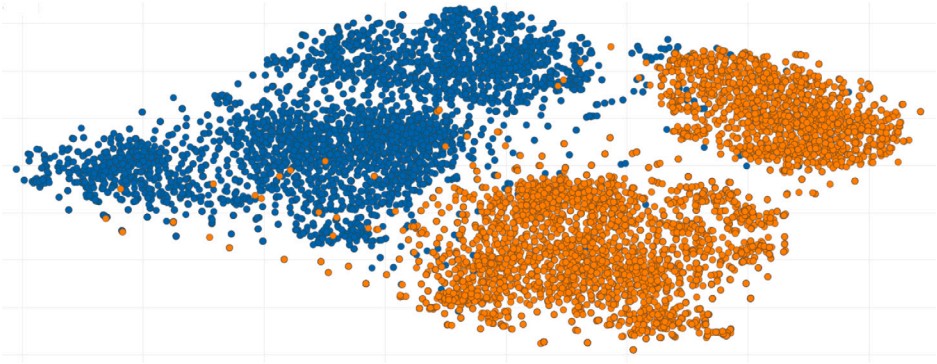


Fig. 3. Visualizing all processed data through a data explorer approach. In this graph, "blue" represents *with_mask* data, while "orange" comprises *without_mask* data. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

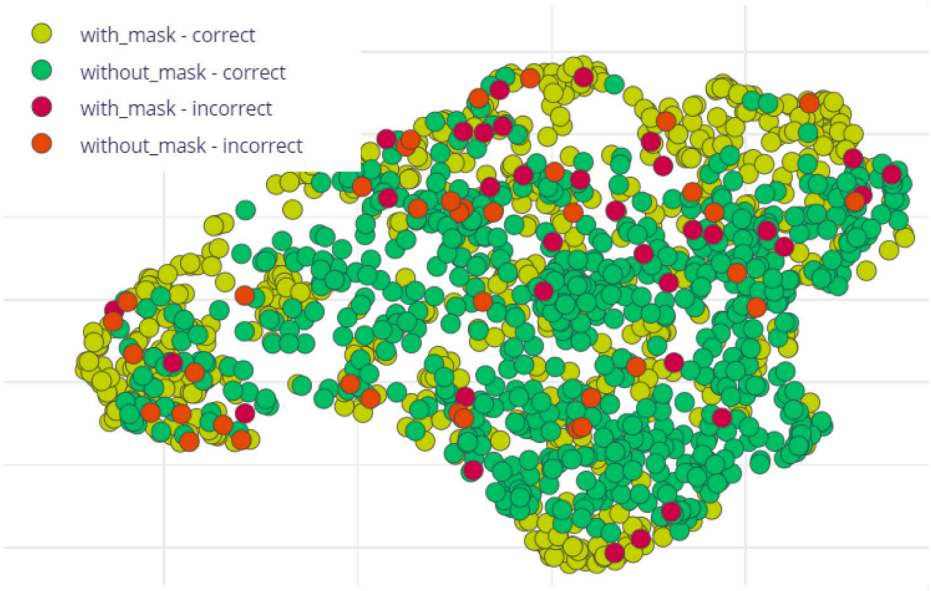


Fig. 4. The achieved results for the testing phase.

The Edge Impulse platform offers a dedicated section for model testing, which performs the prediction of the images from the test base, calculates a group of metrics, and plots a corresponding two-dimensional graph. We used such tools to evaluate the testing phase for the considered test data, as presented in Fig. 4.

Furthermore, some well-known metrics for the evaluation of a classification model could be computed (Pang, Nijkamp, & Wu, 2020). The considered metrics in this case are briefly described as follows:

- Accuracy: this metric calculates how often predictions are equal to their labels, that is, the proportion of correct predictions among the entire considered set;
- Precision: this metric computes the precision of the predictions with respect to the labels, which means that it calculates the proportion of predictions of the "positive" class that are correct;
- Recall: this metric calculates the recall of the predictions with respect to the labels, which is, it calculates how good the model performs to predict the "positive" class;
- F1-score: this metric calculates the F1-score, which is the harmonic mean between the metrics precision and recall, showing the balance between them;
- Confusion matrix: it represents the confusion matrix from predictions and labels, that is, it is a matrix table that shows the correct and wrong predictions of the model, compared to the expected result.

Table 3
Achieved testing metrics. Number of samples used for testing: 1461.

Metric	Value
Accuracy	95%
Precision	96%
Recall	95%
F1-Score	96%

Table 3 presents the metric values for the testing phase of the model. With this information, it is possible to observe that the model not only had a good hit rate for both classes (with and without mask), but also a balanced hit rate. This demonstrates the relevance of balancing the data classes during model training.

Moreover, a confusion matrix can also be created, as presented in Table 4. In that Table, it is possible to see that the employed model had a high hit rate, getting the uncertain result in only a small number of cases.

The graph shown in Fig. 4 allows us to look specifically at which images the model made the incorrect prediction. After some analysis, it was possible to observe that, besides the reasons mentioned previously, the model incorrectly labeled some images where a person was looking at the side and not at the camera, and in situations where the person was wearing glasses, as further observed for the incorrect classification

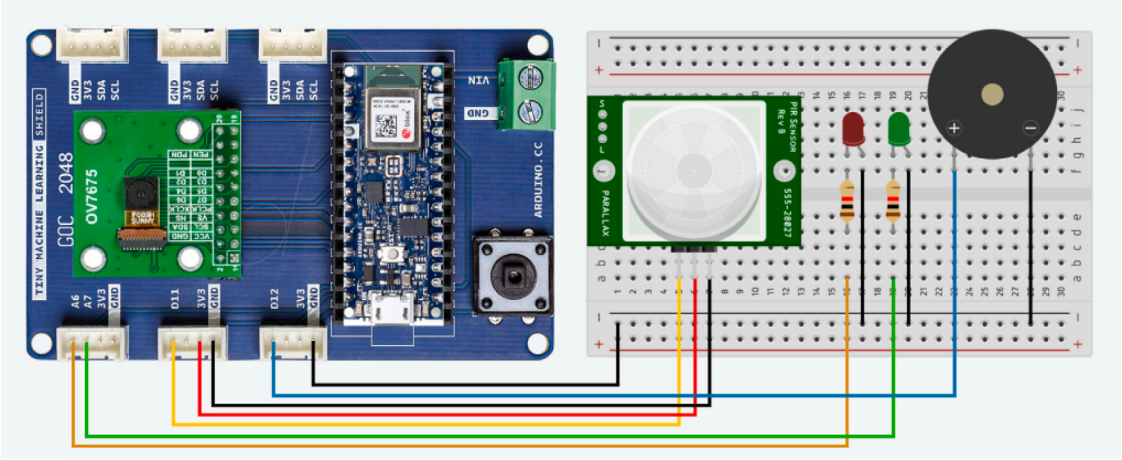


Fig. 5. The assembled monitoring unit to detect face masks.

Table 4
The achieved confusion matrix. Number of samples used for testing: 1461.

		Predicted			
		Samples	With_mask	Without_mask	Uncertain
Actual	With_mask	711	95.2%	3.1%	1.7%
	Without_mask	750	3.9%	94.9%	1.2%

cases. However, for the computed accuracy of 95%, one could expect that the achieved model will have a very reasonable success rate at a very low computational cost.

4.2. Model deployment

After finishing all the construction steps and the initial validation procedures of the TinyML model, a deployment step was performed on the Arduino Nano 33 Sense board, comprising then the last step of the development pipeline described in Fig. 2. This deployment was performed by creating a customized library for the selected Arduino board through the Edge Impulse platform, allowing the creation of optimized code that can be easily customized and integrated.

In order to allow that the correct use of face masks is only assessed when necessary, a HC-SR501 presence sensor was employed in the developed prototype. This sensor has the function to optimize the overall processing and reduce energy consumption, which might be an issue in some deployment areas without a direct energy supply. Additionally, LEDs are used in the prototype to indicate whether the assessed person is wearing a mask (green LED) or not (red LED), along with a buzzer that is triggered when it detects that the user is not wearing a mask, with the purpose of drawing attention to this situation. Overall, all these boards and electronic components were chosen concerning their usability, easy connection, affordability, and availability in usual electronic stores, facilitating quick and low-cost reproduction of our approach.

Fig. 5 represents a diagram with the proposed solution, while Fig. 6 presents the developed prototype assembled according to the designed diagram. It is composed of an Arduino Nano 33 BLE Sense board, an OV7675 camera module, a Tiny Machine Learning shield, two LEDs, one buzzer, and one HC-SR501 sensor. These components were chosen with the aim of proposing an effective, low-cost solution with a good user experience. The LEDs and the Buzzer aim to give a clear indication of the result of the algorithm, and the presence sensor aims to activate the prediction only when there is someone in front of the camera, in order to avoid unnecessary computational costs.

When performing the transformation from Python code using the Tensorflow Lite library to an Arduino C++ library, it was necessary, after all the model training process, to select an option to perform model

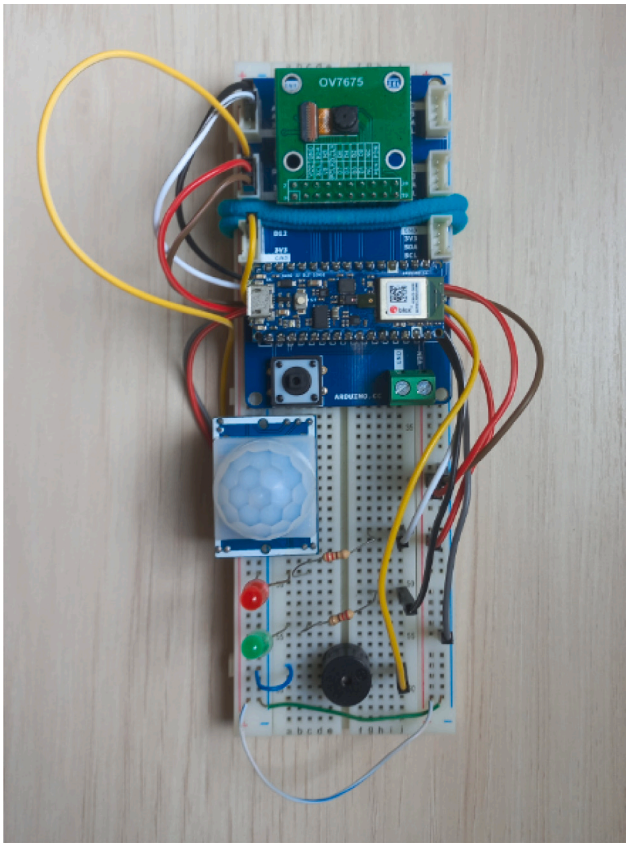


Fig. 6. The assembled monitoring unit to detect face masks.

optimizations. The Edge Impulse platform allows selecting two options, one that does not include optimizations, maintaining the use of float32 variables in the calculations, and another that allows optimizations, more specifically, quantization, using int8 variables in the calculations.

In general, this post-training quantization option works as a compression technique for the model (Liu et al., 2021), reducing its size and improving hardware latency in a way that does not significantly impair the accuracy value of the model. This means that the model was initially trained with an accuracy using floating-point representation with 32 bits, where this representation depends on the signal coding, the significance, and the exponent (Novac, Boukli Hacene, Pegatoquet,

Table 5
Results obtained when using the prototype deployed on the entrance of a research lab at the Federal University of Rio Grande do Norte.

Day	People with mask	People without mask	Correct predictions
Monday	3	5	7 of 8
Tuesday	2	6	6 of 8
Wednesday	1	3	4 of 4
Thursday	1	4	3 of 5
Friday	2	3	4 of 5

Miramond, & Gripon, 2021), and then it was quantized using a fixed-point representation of 8 bits in its calculations so that the numbers are now represented with an integer and a fractional part (Novac et al., 2021).

In the developed prototype, the only option available on the platform was chosen to perform the quantization in the model. Compared to the model without optimizations, it allowed a 60% decrease in the amount of RAM used, occupying a total of 131.3 KB (51.29% of the available 256 KB RAM memory), as well as a 67.4% decrease in the amount of memory flash memory occupied, totaling 323.9 KB (31.63% of the available 1MB Flash memory). In this process, a great improvement was obtained in the amount of memory used, as well as latency, without decreasing the accuracy of the model, in which, compared to the model without optimizations, the final accuracy decreased by less than 0.5%.

The assembled prototype unit was positioned on the entrance of an academic department room at the Federal University of Rio Grande do Norte, Brazil, with its camera facing the door and with a distance of 1.60 m from the ground in a way that it could easily view the face of coming "detected" people.

4.3. Detecting face masks in a real-world scenario

For a total period of five days (from Monday to Friday), we assessed the prototype for a group of people that randomly received face masks for the experiments, since the relaxation of the COVID-19 restrictions strongly reduced the use of masks in 2023. In this experiment, in general, the prototype successfully alerted about the absence of masks and performed well for masks not properly used, which we considered a significant result for a first implementation on an affordable hardware platform.

Table 5 summarizes the achieved results, for a total of 30 people evaluated in the considered experimental setup. In this general scenario, it was found that among the 30 people detected, 24 were correctly classified according to whether they were wearing a mask or not. It could be observed that the incorrect predictions of the model generally referred to people with a thick beard, who were without a mask, and were incorrectly classified as people wearing a mask. In general, it is clear that the model presented a good result for the experiment carried out. Moreover, it is worth noticing that, in order to improve the results and achieve a better number of correct predictions, it is necessary to add a greater number of images of people with thick beards to the database, so that the model can be retrained to correctly classify people with this characteristic. This is, in fact, a common approach for machine learning models and thus we expect that more balanced training datasets would reduce eventual bias and accuracy issues.

Particularly for this scenario, a group of particular situations (correct and incorrect mask usage) were carefully selected to validate the model in each expected configuration, taking for that five individuals of distinct age groups and genders who adopted different mask usage configurations. Individuals of different genders and ages were chosen in order to evaluate the behavior of the model for several facial characteristics, among them, facial hair, features and other aspects that can change according to age and gender. Fig. 7 presents the evaluated mask usage and the achieved performance.

As can be seen in Table 6, the configured TinyML model was effective when detecting the proper use of masks, with a reasonable percentage of successful classifications.

Some considerations can be drawn for the achieved results. First, the developed solution had issues in some situations when the mask was not adequately adjusted, as is the case of its use with the nose uncovered. In addition, it was noticeable that there was difficulty when the test was performed on an individual with a beard, showing an accuracy less than the other scenarios. These limitations happen due to the absence of images with a higher level of complexity in the training database, although the achieved results can be considered very satisfactory for real-world scenarios. Moreover, the characteristics of the adopted compressed model and also the low resolution of the employed camera are also assumed as limiting potentials.

Therefore, after the performed validations with both the test dataset and the images taken from the deployed model on a real microcontroller-based board, we could conclude that the developed approach can be effectively employed for the detection of face masks, strongly supporting when reinforcing social distance measures for the COVID-19 pandemic or any eventual airborne disease outbreak. Moreover, we could also validate the practical application of a TinyML model in a real scenario, which was supported by an extensive literature review and performance comparisons before a transfer learning process could be satisfactorily adopted. Finally, the achieved results were significant when fostering the development of TinyML-based soft-sensor solutions exploiting ML platforms, which can not only save development time but also enhance the overall development cycle for a new series of embedded on-the-edge applications.

4.4. Hardware performance and time constraints

Since we are dealing with a tiny machine learning model that is embedded into a affordable microcontroller, with specific hardware constraints in processing, memory and even data acquisition, some important concerns may arise when deploying the model in a real-world scenario to detect face masks. Actually, although the processing and memory limitations were already addressed by the Edge Impulse platform, as aforementioned, some other characteristics might impair the achievable performance, notably for a detection unit based on the very restricted OV7675 camera module. In this case, issues around lightning and face angles, for instance, are more stringent for such a camera, but could be not a problem for more robust camera hardware depending to the available budget. This way, we focused the hardware performance of the performed experiments on more attainable parameters, based on the actual prototype's hardware assembling.

Therefore, regarding the prototype's execution time, in order to perform the complete process of taking a picture of a person, classify whether she or he is wearing a mask, and alert about the obtained result (LEDs switching), we noticed that an average time of 3.3 s was taken. In practical means, a person that has to be given permission to enter into the area of the research lab in this experiment had to be positioned in front of the prototype, indirectly activate the motion sensor, and wait for 3.3 s until the unit can decide whether a (mandatory) face mask is being properly used. In fact, we considered that the processing time was in accordance with the expected results for the employed hardware, although we agree that it should be lower for large-scale applications.

When analyzing the achieved performance concerning the overall processing time, we could drawn some remarks that should be valuable for future projects based on the TinyML paradigm. First, our microcontroller-based approach is basically centered on affordability and reduced energy consumption, with a lot of potential applications in different scenarios. Nevertheless, particularly for our prototype, the Arduino 33 BLE Sense is not the most powerful MCU and there are some (more expensive) microcontrolers that could be more suitable for a group of TinyML approaches that need to retrieve visual data from cameras in real-time, specially when concerning the processing



Fig. 7. Examples of pictures taken by the OV7675 camera with the situations: (a) without mask; (b) with mask; (c) mask covering only the mouth; (d) mask on the chin; (e) mask on the forehead.

Table 6
Some results for the tests with the deployed prototype unit, considering five different situations. The persons' age and gender are as follows: Person 1: 12 years, Male; Person 2: 22 years, Female; Person 3: 46 years, Female; Person 4: 76 years, Female; Person 5: 50 years, Male.

Situation	Person 1	Person 2	Person 3	Person 4	Person 5
A	with: 0.05 without: 0.95	with: 0.00 without: 1.00	with: 0.01 without: 0.99	with: 0.01 without: 0.99	with: 0.19 without: 0.81
B	with: 1.00 without: 0.00	with: 0.99 without: 0.01	with: 1.00 without: 0.00	with: 1.00 without: 0.00	with: 1.00 without: 0.00
C	with: 1.00 without: 0.00	with: 0.98 without: 0.02	with: 1.00 without: 0.00	with: 1.00 without: 0.00	with: 0.99 without: 0.01
D	with: 0.03 without: 0.97	with: 0.96 without: 0.04	with: 0.42 without: 0.58	with: 0.04 without: 0.96	with: 0.01 without: 0.99
E	with: 0.10 without: 0.90	with: 0.00 without: 1.00	with: 0.09 without: 0.91	with: 0.03 without: 0.97	with: 0.39 without: 0.61

time (Giménez, Freitag, Lee, & Vandierendonck, 2022). Whatever the case, while the developed prototype was intended to demonstrate a low-cost real-world solution based on our proposed development pipeline, other microcontrollers with better resources could be adopted instead when the overall performance was a relevant issue.

Other interesting remark is about the used camera. The OV7675 camera is quite simple and its control hardware does not work in parallel, but in a sequential way, becoming a bottleneck in what concerns the execution time just to take the picture of the subject, which was accounted in 2.5 s in average. On the other hand, the prediction algorithm itself takes about 800 ms to execute after obtaining the image, which is very reasonable for many scenarios.

Finally, an important performance issue is about concurrent execution of multiple tasks. The developed prototype operates as a single sequential thread, but concurrency could be exploited to enhance performance. In this sense, future implementations will consider no only the use of microcontroller boards with better hardware resources, but also the use of real-time operating systems like FreeRTOS.

4.5. Other advanced approaches for face mask detection

Detecting whether people are wearing protective face masks correctly in various settings is crucial for public health and safety. Overall, this can be accomplished by leveraging different advanced technological approaches, potentially enhancing the accuracy and efficiency of this task. In this context, a group of intuitive approaches involves the use of computer vision and machine learning algorithms, as discussed in this article. Cameras placed at entrances or checkpoints can analyze real-time video feeds to identify individuals and assess whether they are wearing masks. These algorithms can distinguish between different types of masks, check if they cover the nose and mouth properly, and even detect mask breaches such as masks worn below the nose or chin. By providing real-time feedback to both individuals and security personnel, this basic group of approaches can help ensure mask compliance in crowded areas like airports, public transportation, or healthcare facilities.

Another promising approach is the integration of wearable technology and Internet of Things (IoT) devices (Mavrogiorgou, Kiourtis, & Kyriazis, 2022). Smart masks embedded with sensors can monitor the wearer's breath and provide feedback on mask fit and seal. These masks can connect to a mobile app or a centralized system to alert wearers of any issues and ensure proper mask usage. Additionally, IoT-based solutions can deploy environmental sensors in indoor spaces to monitor air quality and mask compliance. When individuals enter an area without proper mask usage, automated alerts can be sent to the relevant authorities for immediate action. This approach not only encourages mask-wearing but also helps maintain a safe environment by promptly addressing compliance issues.

Furthermore, the development of contactless technologies, such as thermal imaging and LiDAR (Light Detection and Ranging), can assist in detecting individuals without masks (Taha et al., 2021). Thermal imaging cameras can identify individuals with elevated skin temperature, which may indicate a fever and the need for mask usage. LiDAR sensors, on the other hand, can create 3D maps of an environment and detect whether a person's face is obscured by an object or if they are not wearing a mask. Combining these technologies with automated alerts and access control systems can efficiently manage entry points to ensure mask compliance without direct human intervention.

In conclusion, advanced approaches to detect face masks involve a combination of computer vision, wearable technology, IoT devices, and contactless technologies, and many innovative issues are expected to be developed in the coming years. These methods will not only enhance mask compliance but also contribute to public safety by monitoring and enforcing mask usage in various settings, particularly in crowded

or high-risk areas. Leveraging these technological innovations can play a crucial role in preventing the spread of contagious diseases and protecting public health.

5. Potential applications for the model

The use of embedded electronic devices to gather information from the environment has been largely adopted in a lot of applications lately, specially when considering the reduction of the hardware costs and the increasing of processing power, memory resources, and communication capabilities of microcontroller boards. With the advent of embedded machine learning techniques, different types of artificial intelligence algorithms could be exploited in such applications, opening new possibilities.

The proposed development pipeline, as well as the prototype implementation openly available at Azevedo et al. (2023), can be employed in different applications. Although specific scenarios are not defined, we selected three potential applications that can significantly benefit from our approach:

- Airport security and border control: As passengers move through the security checkpoints or immigration queues one by one, a compliant camera system will analyze each individual's face and determine whether they are wearing a mask properly;
- Hospital control: As patients, visitors, and healthcare workers through different areas, each entrance could employ an affordable camera-based system to detect face masks. This system could go beyond monitoring mask compliance, also controlling physical access by unlocking doors only when the face mask requirements are met;
- Staircases in metro stations: By strategically installing cameras at different points along the staircases, the system can continuously monitor individuals as they ascend or descend the stairs. In such systems, accuracy would potentially increase when more hardware units are adopted, which plays in favor of more affordable camera-based TinyML solutions.

Once again, we expect a large variety of applications that can strongly benefit of TinyML-based solutions when exploiting affordable cameras to make fast decisions. Nevertheless, for the particular case of enforcing face mask compliance to prevent airborne diseases, it is important to note that such systems should be implemented alongside other preventive measures, potentially improving the success rate when preventing virus outbreaks.

In this last aspect, it is worth mentioning that one of the reasons for the dramatic events of the COVID-19 pandemic was the lack of preparedness concerning a myriad of technological tools that might be effective when halting the initial spread of the SARS-CoV-2 virus. When in May 2023 the World Health Organization declared that the COVID-19 was no longer a global health emergency, many lessons were already learned from this tumultuous process that leveraged a lot of IoT and machine learning technologies. Nevertheless, it has become highly anticipated that new airborne disease outbreaks will become more common (Shafaati et al., 2023), and this time technologies should be ready to provide faster responses (Costa & Peixoto, 2020). Overall, this principle should hold even without regularity policies of the countries demanding the use of masks in public spaces. Therefore, flexible, adaptable, and potentially affordable face mask detection approaches, as the one proposed in this work, can be a valuable asset to face new pandemics in the next years.

6. Conclusions

The ability to accurately and efficiently detect masks usage is crucial for public health and safety. With the dramatic events of the COVID-19 pandemic, it became clear that the use of face masks could significantly halt the spread of the virus, which demanded the creation of computational solutions to ensure the correct and constant use of the masks. In this sense, this work comes as a valuable resource when defining a consistent development methodology and discussing the technical fundamentals to implement a practical solution toward that goal, with practical results than can be leveraged for any type of respiratory diseases outbreaks.

The selected model and the performed training achieved a high accuracy level that indicated its effectiveness for practical applications. Moreover, the validation and testing phases successfully confirmed the expected results, further indicating the benefits for its adoption. The conclusion that we can take is that the intended results were achieved, and that the adopted procedures can be easily replicated in other TinyML-based approaches. This way, our approach represents a cutting-edge solution with clear modern relevance in ensuring public health and safety, particularly underscored by the imperative of face mask usage during the COVID-19 pandemic and next global health emergencies. By providing a comprehensive development methodology and robust technical foundation, our work offers a practical avenue for accurate mask detection and usage monitoring, adaptable to various respiratory disease outbreaks.

As future works, new datasets will be considered for both training and tests, with the aim of increasing both the number of samples and covering different situations of mask usage, including masks covering only the mouth or in an inappropriate unsafe way. Additionally, new target microcontroller boards will be exploited, specially the ones with more memory and processing capacities. Finally, ways to integrate this classifier with solutions in the area of computer vision will be studied, proposing to raise the level of the final embedded unit. In this way, connecting what has already been developed with libraries such as OpenCV can result in a new complete system, aggregating the mask classifier hardware with a software that allows real-time visualization of the images and their respective predictions.

CRediT authorship contribution statement

Mariana B. Azevedo: Conceptualization, Methodology, Formal analysis, Writing — original draft, Visualization. **Thaís de A. de Medeiros:** Conceptualization, Software, Investigation, Writing – original draft, Visualization. **Morsinaldo de A. Medeiros:** Conceptualization, Validation, Formal analysis, Writing – original draft, Visualization. **Ivanovitch Silva:** Term, Resources, Writing – review & editing, Supervision, Project administration, Funding acquisition. **Daniel G. Costa:** Data curation, Writing — original draft, Writing – review & editing, Visualization, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is financially supported by INEGI-LAETA, Portugal (FCT project UIDB/50022/2020). It is also supported by national funds through the FCT/MCTES (PIDDAC), Portugal, under the project EXPL/EEI-COM/1089/2021. Finally, this work was also financed by the Brazilian fostering agency CNPq (National Council for Scientific and Technological Development), Process No. 405531/2022-2.

References

Abdullah, & Hasan, M. S. (2017). An application of pre-trained CNN for image classification. In *2017 20th International conference of computer and information technology* (pp. 1–6). http://dx.doi.org/10.1109/ICCITECHN.2017.8281779.

Andrade, P., Silva, I., Silva, M., Flores, T., Cassiano, J., & Costa, D. G. (2022). A tinyml soft-sensor approach for low-cost detection and monitoring of vehicular emissions. *Sensors*, *22*(10), http://dx.doi.org/10.3390/s22103838.

Arduino docs (2023). Nano 33 BLE sense. URL https://docs.arduino.cc/hardware/nano-33-ble-sense.

Azevedo, M., Medeiros, T., Medeiros, M., Silva, I., & Costa, D. (2023). TinyML Face Mask Detection Code. URL https://github.com/conect2ai/facemask_detection.

Bojer, C. S., & Meldgaard, J. P. (2021). Kaggle forecasting competitions: An overlooked learning opportunity. *International Journal of Forecasting*, *37*(2), 587–603. http://dx.doi.org/10.1016/j.ijforecast.2020.07.007, URL https://www.sciencedirect.com/science/article/pii/S0169207020301114.

Budd, J., Miller, B. S., Manning, E. M., Lampos, V., Zhuang, M., Edelstein, M., et al. (2020). Digital technologies in the public-health response to COVID-19. *Nature Medicine*, *26*(8), 1183–1192. http://dx.doi.org/10.1038/s41591-020-1011-4.

Cetinic, E., Lipic, T., & Grgic, S. (2018). Fine-tuning convolutional neural networks for fine art classification. *Expert Systems with Applications*, *114*, 107–118. http://dx.doi.org/10.1016/j.eswa.2018.07.026.

Costa, D. G. (2020). Visual sensors hardware platforms: A review. *IEEE Sensors Journal*, *20*(8), 4025–4033. http://dx.doi.org/10.1109/JSEN.2019.2952447.

Costa, D. G., & Peixoto, J. P. J. (2020). COVID-19 pandemic: A review of smart cities initiatives to face new outbreaks. *IET Smart Cities*, *2*(2), 64–73. http://dx.doi.org/10.1049/iet-smc.2020.0044.

Dargan, S., Kumar, M., Ayyagari, M. R., & Kumar, G. (2020). A survey of deep learning and its applications: A new paradigm to machine learning. *Archives of Computational Methods in Engineering*, *27*, 1071–1092. http://dx.doi.org/10.1007/s11831-019-09344-w.

Denton, E., Hanna, A., Amironesei, R., Smart, A., & Nicole, H. (2021). On the genealogy of machine learning datasets: A critical history of ImageNet. http://dx.doi.org/10.1177/20539517211035955.

Giménez, N. L., Freitag, F., Lee, J., & Vandierendonck, H. (2022). Comparison of two microcontroller boards for on-device model training in a keyword spotting task. In *2022 11th Mediterranean conference on embedded computing* (pp. 1–4). http://dx.doi.org/10.1109/MECO55406.2022.9797171.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. CoRR abs/1704.04861, arXiv:1704.04861, URL http://arxiv.org/abs/1704.04861.

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *2017 IEEE conference on computer vision and pattern recognition* (pp. 2261–2269). http://dx.doi.org/10.1109/CVPR.2017.243.

Hymel, S., Banbury, C., Situnayake, D., Elium, A., Ward, C., Kelcey, M., et al. (2022). Edge impulse: An mlops platform for tiny machine learning. http://dx.doi.org/10.48550/ARXIV.2212.03332, URL https://arxiv.org/abs/2212.03332.

Joseph, V. R. (2022). Optimal ratio for data splitting. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, *15*(4), 531–538.

Kaya, Y., & Gürsoy, E. (2023). A MobileNet-based CNN model with a novel fine-tuning mechanism for COVID-19 infection detection. *Soft Computing*, 1–15. http://dx.doi.org/10.1007/s00500-022-07798-y.

Ker, J., Wang, L., Rao, J., & Lim, T. (2017). Deep learning applications in medical image analysis. *Ieee Access*, *6*, 9375–9389. http://dx.doi.org/10.1109/ACCESS.2017.2788044.

Kumar, A., Kaur, A., & Kumar, M. (2019). Face detection techniques: A review. *Artificial Intelligence Review*, *52*, 927–948. http://dx.doi.org/10.1007/s10462-018-9650-2.

Li, Y., Zhang, H., Xue, X., Jiang, Y., & Shen, Q. (2018). Deep learning for remote sensing image classification: A survey. *WIREs Data Mining and Knowledge Discovery*, *8*(6), Article e1264. http://dx.doi.org/10.1002/widm.1264.

Liu, Z., Wang, Y., Han, K., Zhang, W., Ma, S., & Gao, W. (2021). Post-training quantization for vision transformer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, & J. W. Vaughan (Eds.), *Advances in Neural Information Processing Systems*: *vol. 34*, (pp. 28092–28103). Curran Associates, Inc., URL https://proceedings.neurips.cc/paper/2021/file/ec8956637a99787bd197eacd77acce5e-Paper.pdf.

Mavrogiorgou, A., Kiourtis, A., & Kyriazis, D. (2022). A pluggable IoT middleware for integrating data of wearable medical devices. *Smart Health*, *26*, Article 100326. http://dx.doi.org/10.1016/j.smhl.2022.100326, URL https://www.sciencedirect.com/science/article/pii/S2352648322000605.

Mbunge, E., Akinnuwesi, B., Fashoto, S. G., Metfula, A. S., & Mashwama, P. (2021). A critical review of emerging technologies for tackling COVID-19 pandemic. *Human Behavior and Emerging Technologies*, *3*(1), 25–39. http://dx.doi.org/10.1002/hbe2.237.

Minaee, S., Boykov, Y. Y., Porikli, F., Plaza, A. J., Kehtarnavaz, N., & Terzopoulos, D. (2021). Image segmentation using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, http://dx.doi.org/10.1109/TPAMI.2021.3059968.

Mohan, P., Paul, A. J., & Chirani, A. (2021). A tiny CNN architecture for medical face mask detection for resource-constrained endpoints. http://dx.doi.org/10.1007/978-981-16-0749-3_52.

Nan, Y., Ju, J., Hua, Q., Zhang, H., & Wang, B. (2022). A-MobileNet: An approach of facial expression recognition. *Alexandria Engineering Journal*, *61*, 4435–4444. http://dx.doi.org/10.1016/j.aej.2021.09.066.

Novac, P.-E., Boukli Hacene, G., Pegatoquet, A., Miramond, B., & Gripon, V. (2021). Quantization and deployment of deep neural networks on microcontrollers. *Sensors*, *21*(9), http://dx.doi.org/10.3390/s21092984, URL https://www.mdpi.com/1424-8220/21/9/2984.

Ojo, M. O., Giordano, S., Procissi, G., & Seitanidis, I. N. (2018). A review of low-end, middle-end, and high-end iot devices. *IEEE Access*, *6*, 70528–70554. http://dx.doi.org/10.1109/ACCESS.2018.2879615.

Pang, B., Nijkamp, E., & Wu, Y. N. (2020). Deep learning with TensorFlow: A review. *Journal of Educational and Behavioral Statistics*, *45*(2), 227–248. http://dx.doi.org/10.3102/1076998619872761.

Pham, H. T., Nguyen, M. A., & Sun, C. C. (2019). AIoT solution survey and comparison in machine learning on low-cost microcontroller. In *2019 International symposium on intelligent signal processing and communication systems* (pp. 1–2). http://dx.doi.org/10.1109/ISPACS48206.2019.8986357.

Piatkowski, D., & Walkowiak, K. (2022). TinyML-based concept system used to analyze whether the face mask is worn properly in battery-operated conditions. *Applied Sciences*, *12*(1), http://dx.doi.org/10.3390/app12010484, URL https://www.mdpi.com/2076-3417/12/1/484.

Raza, W., Osman, A., Ferrini, F., & Natale, F. D. (2021). Energy-efficient inference on the edge exploiting TinyML capabilities for UAVs. *Drones*, *5*(4), 127. http://dx.doi.org/10.3390/drones5040127.

Revina, I., & Emmanuel, W. S. (2021). A survey on human face expression recognition techniques. *Journal of King Saud University - Computer and Information Sciences*, *33*(6), 619–628. http://dx.doi.org/10.1016/j.jksuci.2018.09.002.

Said, Y. (2020). Pynq-YOLO-net: An embedded quantized convolutional neural network for face mask detection in COVID-19 pandemic era. http://dx.doi.org/10.14569/IJACSA.2020.0110912, URL https://thesai.org/Downloads/Volume11No9/Paper_12-Pynq_YOLO_Net_An_Embedded_Quantized.pdf.

Sarker, L., Islam, M. M., Hannan, T., & Ahmed, Z. (2020). COVID-DenseNet: A deep learning architecture to detect COVID-19 from chest radiology images. http://dx.doi.org/10.20944/preprints202005.0151.v1, URL https://www.preprints.org/manuscript/202005.0151/v1.

Seker, M., Männistö, A., Iosifidis, A., & Raitoharju, J. (2022). Automatic social distance estimation for photographic studies: Performance evaluation, test benchmark, and algorithm. *Machine Learning with Applications*, *10*, Article 100427. http://dx.doi.org/10.1016/j.mlwa.2022.100427, URL https://www.sciencedirect.com/science/article/pii/S2666827022001025.

Shafaati, M., Chopra, H., Priyanka, Khandia, R., Choudhary, O. P., & Rodriguez-Morales, A. J. (2023). The next pandemic catastrophe: Can we avert the inevitable? *New Microbes and New Infections*, *52*, Article 101110. http://dx.doi.org/10.1016/j.nmni.2023.101110, URL https://www.sciencedirect.com/science/article/pii/S205229752300029X.

Sharifi, A., Zibaei, A., & Rezaei, M. (2021). A deep learning based hazardous materials (HAZMAT) sign detection robot with restricted computational resources. *Machine Learning with Applications*, *6*, Article 100104. http://dx.doi.org/10.1016/j.mlwa.2021.100104.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. http://dx.doi.org/10.48550/ARXIV.1409.1556.

Suárez-Cetrulo, A. L., Quintana, D., & Cervantes, A. (2023). A survey on machine learning for recurring concept drifting data streams. *Expert Systems with Applications*, *213*, Article 118934. http://dx.doi.org/10.1016/j.eswa.2022.118934, URL https://www.sciencedirect.com/science/article/pii/S0957417422019522.

Taha, B. A., Al Mashhadany, Y., Bachok, N. N., Ashrif A. Bakar, A., Hafiz Mokhtar, M. H., Dzulkefly Bin Zan, M. S., et al. (2021). Detection of COVID-19 virus on surfaces using photonics: Challenges and perspectives. *Diagnostics*, *11*(6), http://dx.doi.org/10.3390/diagnostics11061119.

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning research*, *9*, p. 2579.

Wang, Z., Chen, J., & Hoi, S. C. (2020). Deep learning for image super-resolution: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *43*(10), 3365–3387. http://dx.doi.org/10.1109/TPAMI.2020.2982166.

Warden, P., & Situnayake, D. (2020). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (illustrated ed.). (p. 484). O'Reilly.

Yasin Kaya, E. G. (2023). A MobileNet-based CNN model with a novel fine-tuning mechanism for COVID-19 infection detection. *Soft Computing*, http://dx.doi.org/10.1007/s00500-022-07798-y.

Zafeiriou, S., Zhang, C., & Zhang, Z. (2015). A survey on face detection in the wild: Past, present and future. *Computer Vision and Image Understanding*, *138*, 1–24. http://dx.doi.org/10.1016/j.cviu.2015.03.015.

Zhu, Y., & Newsam, S. D. (2017). DenseNet for dense flow. In *2017 IEEE International Conference on Image Processing (ICIP)* (pp. 790–794). http://dx.doi.org/10.1109/ICIP.2017.8296389.

View publication stats